

Sample More, Reflect Less

Self-Refine and Reflexion Lose to Repeated Sampling at Equal Token Cost, from 1.5B to 7B

Iliya Mirzaei
Department of Computer Science
Stony Brook University
imirzaei@cs.stonybrook.edu

July 31, 2026

Abstract

Many methods promise to make a language model reason better without retraining it. The model is told to plan before it solves, to criticise and rewrite its own answer, to reflect on its mistakes, to pick the best of several attempts, or to argue with copies of itself. Nearly all of these methods also make the model write far more text than a single chain of thought, and writing more text raises accuracy by itself. So a gain over one chain of thought does not show that the method’s idea is what helped.

Wang et al. [27] made this point and reported that a simple baseline often wins once the budgets are made comparable. That baseline is to ask the same question several times and keep the answer that comes up most often. Their evidence is a set of point estimates, with no confidence intervals, no significance tests, and 100 questions per dataset.

We run the comparison again as a designed experiment: seven methods, open models of 1.5B, 3B and 7B parameters, two mathematics benchmarks, 150 questions each. We count every token a method generates, including the tokens spent on critiques, reflections, debate turns and checking, and compare each method against repeated sampling at that method’s own measured cost. All 36 comparisons are paired question by question and reported with confidence intervals and a correction for testing many methods at once.

No method is reliably better than repeated sampling at equal cost in any setting. Ten are reliably worse, all of them methods in which the model inspects its own output, and two of those survive a correction applied across all 36 comparisons at once. Every one of the 18 comparisons involving self-inspection comes out negative.

The interesting part is what happens as the model grows, because the two kinds of self-inspection part company. *Choosing* among your own samples stops being harmful: Best-of- N draws eight samples and asks the model to pick the best, and simply counting the most common answer instead is better by 8.0 and 11.3 points at 1.5B but only 2.0 and 1.3 points at 7B, where the difference is no longer distinguishable from zero. *Rewriting* does not recover: Self-Refine and a forced version of Reflexion remain significantly below the equal-cost baseline at 7B, by 3.6 to 10.1 points. Spending tokens to reconsider an answer is a worse use of them than spending the same tokens on another attempt, at every size we can test.

We also found that Reflexion, implemented as its authors describe it, never once triggered its own retry on the smallest model. It judged itself correct on every question, quietly turned into a single chain of thought, and scored well because it had become cheap. A method that decides for itself when to act can stop acting without any outward sign. We release the code, the prompts, every generation, and the scripts we used to check our own numbers.

1 Introduction

A large family of methods claims to make language models reason better without changing the model itself. The model is asked to think step by step [14, 30], to plan before it solves [28], to break the problem into smaller ones [36], to criticise and rewrite its own answer [16], to reflect on its mistakes and try again [23], or to argue with copies of itself until they agree [5].

These methods have something in common that is easy to overlook: almost all of them make the model generate *more text*. A single chain of thought might be 300 tokens. Three rounds of self-criticism might be 1,600. A debate between three copies of the model over two rounds might be 2,500. The method is not the only thing that changed. The compute budget changed too, often by a factor of five or ten. And there is a much simpler way to spend a larger budget: ask the model the same question several times and keep the answer it gives most often [29].

This problem is known. Wang et al. [27] made exactly this argument and tested it. Evaluating seven strategies across six datasets, they concluded that “when we provide a simple baseline like chain-of-thought self-consistency with comparable compute resources, it frequently outperforms reasoning strategies proposed in the literature.” We take that claim as our starting point rather than our contribution.

But it has never been tested statistically. The evidence for that conclusion consists of point estimates. The study reports no confidence intervals, no error bars, no variance across runs, and no significance tests, and each dataset is represented by 100 sampled questions.

It is worth being concrete about what that sample size buys, using our own measurements rather than an assumed variance. Across our 28 comparisons at 150 questions, the standard error of a paired difference in accuracy runs from 1.3 to 3.2 percentage points. Scaled to $n = 100$ that is roughly 1.6 to 3.9 points, so a 95% interval would be about ± 3 to ± 8 points wide. It is wider still if the comparison is not paired on questions, which costs roughly a further factor of $\sqrt{2}$. Many of the differences at stake are smaller than that. A conclusion of the form “the simple baseline frequently wins” is therefore exactly the kind of claim that can be produced by noise when several methods are compared without correction, and whether it survives proper inference is unknown.

What we do. We re-run the comparison as a designed experiment. For seven methods, two benchmarks, and two models, we measure accuracy and exact token cost, compare each method against self-consistency *at that method’s own measured cost*, and report every difference with a paired bootstrap confidence interval and a p -value corrected for testing several methods at once. We also extend the question to a regime the earlier study did not reach: models of 1.5B and 3B parameters, well below its smallest (7B). This matters because the methods under test rely on the model judging its own work, and small models are worse at that. But repeated sampling also needs variety among the answers, which small models may lack, so the direction of the effect is not obvious in advance.

What we find. No method beats repeated sampling at equal cost in any of our four settings, and six comparisons are significantly worse. But the useful result is not the tally. Sorting the methods by *what they buy with the extra tokens* separates them cleanly: every method in which the model is asked to assess or rewrite its own output falls below the cost-matched baseline in all twelve of its comparisons, while methods that add no self-assessment sit on the baseline.

One method lets us test that explanation without any confound at all. Best-of- N draws eight samples and then asks the model to pick the best one. We can take those *same eight samples* and simply count which answer occurs most often. The samples, the tokens, and the model are identical; only the final step differs. Counting wins in every setting at these sizes, by between 5 and 17 percentage points.

Because that comparison is cheap, we could also ask where it stops holding. We ran it on a 7B model as well, and there the penalty falls to about two points and is no longer distinguishable from zero. The claim we end up with is therefore a bounded one: choosing is worse than counting below 7B, and the two draw level at 7B.

Contributions.

1. A controlled demonstration that self-evaluation, not extra computation, is where these methods lose: with the sampled candidates held fixed, a model-chosen answer is worse than a counted one (Section 5).
2. A statistical re-examination of the budget-matched comparison: paired bootstrap intervals, Holm correction across methods, and an explicit statement of the effect size the design can detect (Sections 4 and 5).
3. A finer budget-matching procedure. Rather than fixing a common query or token cap for all methods, we measure each method’s actual cost and compare it against the baseline curve interpolated at that cost (Section 2).
4. An efficient construction of the entire self-consistency accuracy-versus-cost curve from a single pool of samples, which is what makes per-method matching affordable (Section 2).
5. Evidence from the small-model regime (1.5B and 3B), which the prior study does not cover.
6. A complete open release: harness, prompts, and every raw generation, so that answer extraction and grading can be re-checked rather than trusted. The prior study released none of these.

Scope. This is a replication with extensions, not a new phenomenon. Our contribution is to establish how much of the earlier conclusion is supported once uncertainty is quantified, and whether it holds for small open models. We study mathematical reasoning with automatically checkable answers on CPU hardware; we do not test frontier models, and we say plainly in Section 6 what that does and does not license.

2 The idea: compare at equal cost

2.1 Why the usual comparison is unfair

Take Self-Refine as an example. The model writes an answer, then criticises its own answer, then rewrites it. With three rounds of criticism that is seven calls to the model instead of one. If the rewritten answer is better than the first answer, we have learned something, but not necessarily that criticising helps. We have spent seven times the compute, and spending more compute is known to help on its own.

The fair question is: *if we had simply spent those seven calls on seven independent attempts and taken the most common answer, would we have done better or worse?* That is the comparison this paper makes.

2.2 The baseline: just sample more

Our reference point is self-consistency [29]. It is the simplest possible way to convert extra compute into extra accuracy: draw N independent chains of thought at a non-zero temperature, read the final answer off each one, and return whichever answer appears most often. It involves no critique, no planning, no communication between attempts, and no extra prompt engineering.

Because self-consistency can be run at any N , it does not give a single number. It gives a *curve* of accuracy against cost. That curve is what we compare against. We call it the *sampling baseline*. A method is worth its cost only if it lands *above* this curve.

2.3 Measuring cost

We measure cost as the number of tokens the model generates, summed over every call a method makes for a single question. This includes tokens that never appear in the final answer: critiques, self-evaluations, reflections, messages exchanged between debating agents, and verification steps. Those tokens are generated, so they are paid for.

We count generated (completion) tokens rather than wall-clock time because time depends on the hardware, the batch size, and how much of the work can be run in parallel, none of which are properties of the method itself. Section 6 discusses what changes if one counts input tokens or latency instead.

2.4 Getting the whole baseline curve cheaply

Running self-consistency separately at each N we care about would mean generating a fresh batch of chains for every one of them. Instead we generate a *pool* of $K = 16$ independent chains once per question, and then read off self-consistency at any $N \leq K$ by repeatedly drawing N chains from the pool without replacement and taking the majority vote. We evaluate at $N \in \{1, 2, 3, 4, 6, 8, 12, 16\}$, averaging over 200 draws at each value. Each draw yields both an outcome (was the majority answer correct?) and a cost, the summed length of exactly those N chains, and we read the two off the *same* draw, so every point on the curve describes a run that could actually have happened.

This is not an approximation of a different experiment: drawing N chains from a pool of K independent chains is distributed exactly as running self-consistency at N would be. Reusing the pool costs us only Monte-Carlo error from using 200 draws instead of all $\binom{K}{N}$ of them, which we quantify in Section 5.

3 Related work

Methods that spend more at test time. Chain-of-thought prompting showed that asking a model to write out intermediate steps improves reasoning [30], and that a single instruction is often enough to trigger it [14]. Self-consistency samples several chains and takes a majority vote [29]. Plan-and-Solve asks the model to write a plan first [28], and least-to-most decomposes a problem into easier sub-problems [36]. Self-Refine has the model critique and rewrite its own output [16], and Reflexion adds a memory of past failures [23]. Tree of Thoughts searches over partial solutions [32], Graph of Thoughts generalises that search to a graph [1], progressive-hint prompting feeds previous answers back as hints [34], and multi-agent debate runs several copies of the model against each other [5]. All of these increase the number of tokens generated per question, by factors ranging from roughly two to well over ten.

Doubts about self-correction. The closest prior result to ours concerns self-correction specifically. Kamoi et al. [12] survey the area and find that reported gains often depend on conditions that do not hold in practice, such as access to reliable external feedback. Huang et al. [11] showed that when a model is not told whether its answer was right, asking it to correct itself often makes things *worse*, and that reported gains frequently come from using the correct answer to decide when to stop. Our study is broader in scope, covering planning, debate, and verification as well as self-correction, and it differs in what it controls for: Huang et al. control the *information* available to the method, whereas we control the *budget* it consumes. The two controls are complementary, and we adopt theirs as well by never revealing ground truth to any method.

Budget-aware evaluation: the work we replicate. Wang et al. [27] is the direct predecessor of this paper and the source of the claim we test. They evaluate seven strategies (chain-of-thought self-consistency, multi-agent debate, Reflexion, Plan-and-Solve, least-to-most, progressive-hint prompting, and Tree of Thoughts) on six datasets including GSM8K and MATH, using GPT-3.5, GPT-4, Mistral-7B, LLaMA-2-70B, and Mixtral-8x7B. They define budget as input plus output tokens and impose a shared cap of 20 queries or 10k tokens, and report that self-consistency “consistently beat other reasoning strategies across all 5 datasets with significantly less budget.”

We differ in four ways, none of which concern the idea of matching budgets, which is theirs. First, they report point estimates only, with no confidence intervals, error bars, run-to-run variance or significance tests, on 100 questions per dataset, which is not enough to separate differences below roughly ten percentage points from noise; we quantify the uncertainty. Second, their budget matching is a shared cap applied to every method, so methods are compared at whatever cost the cap happens to induce; we measure each method’s realised cost and compare it against the baseline curve interpolated at exactly that cost. Third, their smallest model has 7B parameters, leaving the small-model regime untested. Fourth, they release neither code nor generations, so their grading cannot be audited; we release both.

What is already known about verifiers, and what is not. Our sharpest result concerns Best-of- N , so we should be careful about how much of it is new. It is established that selecting among samples with an imperfect verifier has limits. Stroebl et al. [25] show that resampling only keeps paying off if the verifier is perfect, because an imperfect one lets false positives through, and that this bounds how far Best-of- N can go. It is also reported that selection with external or trained reward models often fails to beat plain majority voting. Verifiers that are trained for the job are a different matter: process supervision [15] and verifiers trained alongside the generator [10] both help, and Zhang et al. [33] find that a trained self-verifier beats majority voting at 7B. Cheaper selection signals that avoid a second model, such as the generator’s own confidence [13], sit between these cases.

Our case is the one in between, and it is the one people actually deploy: the same model, with no training and no reward model, asked in a single prompt to pick the best of its own samples. We are not aware of a published measurement of that specific comparison against majority voting on identical samples, and we provide one. But we want to be plain that the direction is what the verifier literature would predict. The contribution is the size of the gap, the fact that it holds in every setting we test, and the fact that it is measured with the samples held fixed so that nothing else can explain it. Taken with Zhang et al. [33], the natural reading is that the training, not the verifying, is what makes a verifier useful.

Related negative results on debate. Choi et al. [3] show theoretically and empirically that

simultaneous-update debate forms a martingale on agents’ belief in the correct answer, implying no expected gain beyond what majority voting already provides, and report that majority voting accounts for most of the measured benefit of multi-agent debate. Tran et al. [26] reach a compatible conclusion for multi-hop question answering under matched “thinking token” budgets. Our debate results should be read as an independent check of these findings in a different regime rather than as a new discovery.

Cost-matched comparison elsewhere. Outside of audits, comparing at equal compute appears mostly *inside* papers proposing a new method, as a way of showing that the new method beats self-consistency at the same budget [22]. A related line treats the budget as the object of study rather than as a control: Snell et al. [24] ask how best to allocate a fixed test-time budget, Wu et al. [31] fit scaling laws for compute-optimal inference, Brown et al. [2] show how far accuracy rises with repeated sampling alone, and Muennighoff et al. [19] obtain strong results from a deliberately simple budget-forcing scheme. Our baseline is the plainest member of that family, used here as a control rather than as a proposal.

Systematic evaluations of prompting. Preet et al. [20] evaluate eight prompting techniques across ten multiple-choice benchmarks and report that plain baseline prompting matches or beats more elaborate techniques in most configurations. Their study and ours point in a similar direction from different angles: they vary the *wording* of a single-shot prompt on multiple-choice questions, while we vary the *amount of computation* for multi-step and multi-sample methods on open-ended answers. Neither subsumes the other.

Statistics in evaluation. Miller [17] argue that language-model evaluations are experiments and should be reported with error bars and significance tests, which is often not done. We follow that advice: every number we report has a confidence interval, all comparisons are paired on questions, and we correct for testing several methods at once.

4 Experimental setup

4.1 Models

We use two instruction-tuned open-weight models from the Qwen2.5 family [21]: Qwen2.5-1.5B-Instruct and Qwen2.5-3B-Instruct. Both are run with `llama.cpp` [7] at Q8_0 quantisation (8 bits per weight) on CPU. All generations come from a single machine with one AMD EPYC 7452 (32 cores, 64 threads) and 125 GB of RAM; there is no GPU. Section 6 discusses what quantisation and model scale mean for how far these results generalise.

4.2 Benchmarks

We use two mathematical reasoning benchmarks whose answers can be graded automatically and exactly:

- **GSM8K** [4]: grade-school word problems with a single integer answer. We draw 150 questions from the 1,319 test items.
- **MATH-500** [8]: a 500-problem subset of competition mathematics, with answers written as formulas such as $(3, \frac{\pi}{2})$. We draw 150 of them.

Method	Configuration	Model calls per question
Chain-of-Thought [30]	greedy, one sample	1
Plan-and-Solve [28]	greedy, one sample	1
Self-Refine [16]	$R = 3$ critique/revise rounds	7
Reflexion [23]	$R = 3$ rounds, early stop	1 to 10
Reflexion (forced)	$R = 3$ rounds, no early stop	7
Best-of- N + self-verify	$N = 8$, model picks	9
Multi-Agent Debate [5]	$A = 3$ agents, $R = 2$ rounds	9
<i>Sampling baseline</i> [29]	pool of $K = 16$, subsampled	1 to 16

Table 1: The methods under test. Reflexion’s cost varies because it stops early when the model judges its own answer to be correct.

This is half again as many questions per dataset as the 100 used by Wang et al. [27], which matters because the size of that sample is one of the reasons their conclusion could not be tested statistically.

Both samples are drawn once with a fixed random seed and then held constant across every method and model, so all comparisons are on identical questions. We deliberately avoid multiple-choice benchmarks, where a method can score above chance without producing a correct derivation.

4.3 Methods compared

Table 1 lists the seven methods and the number of model calls each makes. We implement each method from the description in its original paper, with one uniform restriction: *no method is ever shown the correct answer*. This matters for the self-correcting methods, where using ground truth to decide when to stop would leak the answer into the procedure [11].

Why there are two versions of Reflexion. Reflexion as specified decides for itself when to stop: after each attempt the model is asked whether its own answer is correct, and it reflects and retries only if it says no. How often that happens turned out to depend heavily on the model. On Qwen2.5-1.5B the answer was “correct” on *every single question* in both benchmarks, so the loop always exited immediately and the method silently collapsed into one chain of thought. On Qwen2.5-3B it fires more often, and on MATH-500 it fires most of the time; the exact rates are in Section 5.

Measuring only this version would therefore tell us little about whether reflection helps, because on the smaller model reflection never happened at all. We also run a *forced* variant that skips the self-assessment and always performs its three reflect-and-retry rounds.

We want to be exact about what each version licenses, because it would be easy to criticise a method we had altered. The first version is Reflexion as Shinn et al. [23] describe it, and it is the only one of the two that supports any claim about their method. The forced variant is *ours*, not theirs. It exists to answer a different question: given that the published procedure did nothing on the smaller model, does the reflect-and-retry mechanism help when it is made to run? Statements about “Reflexion” in this paper refer to the published version. Statements about reflection as a mechanism refer to the forced one, and we label it as such everywhere it appears.

Deterministic methods (Chain-of-Thought, Plan-and-Solve, Self-Refine, and both versions of Reflexion) are run at temperature 0. Methods that rely on diversity between samples (the sampling baseline, Best-of- N , Debate) are run at temperature 0.7. Generation is capped at 1,024 tokens per call for every method equally. Each question–method pair gets a seed derived deterministically

from the model, dataset, method, configuration, and question index, so the entire study can be reproduced exactly.

4.4 Grading

Answers are extracted by a deterministic parser: the content of the last `\boxed{...}` if present, otherwise the text following “the answer is”, otherwise the last number in the response. Extracting the braced content requires real brace matching rather than a regular expression, because answers such as $\frac{3\sqrt{3}}{4}$ nest braces to arbitrary depth and a fixed-depth pattern silently returns nothing. For GSM8K the extracted value is compared numerically with a tolerance of 10^{-6} . For MATH-500 we normalise common LaTeX variants (`\left/\right`, `\dfrac` versus `\frac`, degree symbols, surrounding braces) before comparing as strings, and additionally accept a numeric match.

Validating the grader. A grading bug depresses every method equally and is therefore invisible in a comparison, so we test the grader directly. We construct, for each of the 1,319 GSM8K and 500 MATH-500 test items, a synthetic response that states the gold answer inside `\boxed{...}`, under six cosmetic variations: verbatim, extra spacing before control sequences, `\dfrac` substituted for `\frac`, parentheses wrapped in `\left/\right`, a trailing period, and an extra enclosing brace pair. The parser recovers and grades the answer correctly in 500/500 MATH-500 items and 1,319/1,319 GSM8K items under *all six* variations. Any residual failures in the real experiment are therefore attributable to the model, not to the grader.

The identical grading code is used by the experiment runner and by the analysis. We store the raw text of every generation and re-grade from that text at analysis time, so runner-time and analysis-time grading cannot diverge, and so a reader who disagrees with our parser can substitute their own without regenerating anything. We report the observed rate of unparseable model answers in Section 5.

4.5 Statistics

Every comparison is paired: method and baseline are evaluated on the same 150 questions, so we compare per-question outcomes rather than two independent accuracy numbers.

For a method with measured mean cost c , we locate the point on the sampling baseline curve with the same cost. Because the baseline is only defined at integer N , we linearly interpolate between the two values of N whose mean costs bracket c , in (cost, accuracy) space.

Confidence intervals come from a paired bootstrap [6] over questions with 10,000 resamples: questions are the unit of resampling, since they are what we wish to generalise over. We report the mean difference in accuracy, its 95% interval, and a two-sided p -value. Testing seven methods at once makes it likely that one of them looks unusual by chance, so within each setting the p -values are adjusted using the Holm–Bonferroni procedure [9].

4.6 What we can and cannot detect

This is the question the prior study did not ask, so we state the answer plainly. With 150 paired questions, the standard error of a paired difference in accuracy across our 28 comparisons runs from 1.3 to 3.2 percentage points, with a median of 2.6; the corresponding 95% intervals are between ± 2.6 and ± 6.3 points wide. The design can therefore identify differences of roughly five to six percentage points or larger in the typical case; smaller true differences cannot be reliably separated from zero.

Two consequences follow, and we hold to both in Section 5. First, a result of “no significant difference” means the data are consistent with anything inside the interval. It is not evidence that the true effect is zero. We therefore always report the interval, and describe what magnitude of benefit it still permits. Second, because seven methods are compared in each setting, some difference will look large by chance; the Holm correction is what keeps that from being read as a finding.

4.7 Threats to validity

We list the design choices most likely to affect the conclusions, and what we did about each.

Temperature is confounded with method. The deterministic methods run at temperature 0, while the sampling baseline requires a non-zero temperature to produce diverse samples. A difference between them could therefore reflect temperature rather than method. Our design contains the control for this: self-consistency at $N = 1$ is a single sample at temperature 0.7 and costs almost exactly what one greedy chain of thought costs. Comparing greedy chain-of-thought against SC@1 isolates the temperature effect from everything else, and we report it separately in Section 5.

Interpolating the baseline curve. Self-consistency is defined only at integer N , so matching a method’s cost exactly requires interpolating between two adjacent values of N . We interpolate linearly in (cost, accuracy) space. Over this range the baseline’s accuracy rises quickly at first and then flattens, so a straight line drawn between two points on it sits *below* the true curve. That makes the baseline slightly weaker than it really is. This biases our comparison in favour of the methods under test, not against them.

We measure question variance, not run-to-run variance. This is the most important limitation of our statistics and we want it stated before the results rather than after. Our bootstrap resamples *questions*, so every interval answers “how much would this differ on another sample of questions from the same benchmark”. It does not answer “how much would this differ if the same questions were run again with different random seeds”. Three of our methods involve randomness: the sampling baseline, Best-of- N and debate. Each was executed once, so their seed-level variance is absorbed silently rather than estimated. The deterministic methods are run at temperature 0 and repeat exactly, so for them the question is moot. Since Miller [17], whose advice we otherwise follow, recommends accounting for both sources, our intervals should be read as lower bounds on total uncertainty for the three methods that involve randomness. The direction of that bias is towards *over*-stating significance, so the six significant results should be treated as the least secure part of our findings. The Best-of- N comparison in Section 5 is not affected, because there the samples are held fixed and only the selection rule changes.

Cost matching at the cheap end. Greedy chain-of-thought turns out to cost marginally less than a single sampled chain, 292 against 297 tokens on one setting and similarly elsewhere, because greedy decoding produces slightly shorter solutions. The baseline curve does not extend below $N = 1$, so in three of the four settings chain-of-thought is compared against SC@1 rather than against a cheaper interpolated point. The mismatch is at most five tokens out of roughly three hundred and cannot affect any conclusion, but it means the comparison for that one method is against a baseline costing very slightly more. At the expensive end no such issue arises: no method costs more than SC@16 in any setting, so nothing is extrapolated beyond the measured curve.

Reflexion’s budget is chosen by the method. Reflexion decides for itself how many rounds to run, so the cost we match it at is a cost it selected rather than one we imposed. This is the right comparison for a method one would actually deploy, but it does mean that Reflexion’s position on the cost axis is an outcome of the experiment and not a design parameter. The forced variant, whose budget is fixed in advance, does not have this property.

Monte Carlo error from subsampling. Each point on the baseline curve is estimated by drawing 200 subsets from the pool rather than enumerating all $\binom{K}{N}$ of them. This adds a small amount of noise that our question-level bootstrap does not capture. We quantify its size in Section 5 by re-running the analysis with a different random seed and reporting how much the estimates move.

How wide we cast the correction. Testing seven methods at once makes it likely that one looks unusual by chance, so we apply the Holm correction across the seven methods within each model and dataset. That controls the chance of *any* false alarm inside a setting. A stricter reading would correct across all four settings at once. We report both, so that neither choice has to be taken on trust.

Is the verifier’s prompt doing the damage? Best-of- N carries our sharpest claim, so the obvious objection is that we simply asked the model to judge badly. Three things limit that reading, and one does not.

First, the verifier is not being asked to do anything subtle. It sees the problem and eight complete solutions and is asked which is most likely correct. Second, it is not answering at random: it agrees with the majority answer on 63 to 89 per cent of questions, so it is tracking something real, and it loses ground specifically on the questions where it departs from the majority. Third, on the smaller model it picks the first candidate about three-quarters of the time, which is a position bias rather than a reasoning failure. A preference for whichever option is presented first is a documented property of language models asked to choose between candidates [35], not an artefact of our wording.

What we cannot rule out is that a longer or more structured rubric, or a verifier prompted to score each candidate separately rather than choose among them, would do better. Our claim is therefore about the cheap self-check that Best-of- N as usually described implies, not about every possible way of asking a model to evaluate. A reader who wants to test a better rubric can do so against our stored candidates without regenerating them.

Where does debate belong? We describe debate as ending in a count rather than a judgement, because its final step is a majority vote over the agents. That is not the whole story: each agent also reads and revises against the others, which is a form of mutual assessment. Debate therefore sits between our two groups, and it behaves that way, losing less to the baseline than the self-assessing methods but more than the single-pass ones. We flag this because a clean two-way split would be tidier than the evidence warrants.

Prompt wording, and Reflexion in particular. Each method is one implementation of a description in prose, and prompt wording matters. This bears hardest on our finding that Reflexion’s self-assessment never fires on the smaller model. That behaviour is a property of the model’s response to *our* judging prompt, not a theorem about Reflexion: a differently worded question might elicit “incorrect” more often. We report the exact wording in Appendix A so the claim can

be checked rather than believed, and the forced variant exists precisely so that a conclusion about the reflection mechanism does not rest on the judging prompt at all.

One configuration per method. Each method has settings such as rounds, agents and samples that we fixed in advance rather than tuning. Tuning them per dataset would raise every method’s accuracy, but choosing the best configuration requires the correct answers, which is the leak that Huang et al. [11] identify. Fixing configurations in advance avoids that leak at the cost of possibly under-representing each method’s best case. The same fixed configuration is used for both models and both datasets.

Cost measure, and what happens if we charge for input. We recorded generated tokens only. That was a deliberate choice, because generated tokens are the part of the cost that does not depend on how a deployment caches or batches prompts, but it is also the accounting under which the methods we test look *best*, so it deserves a direct answer rather than an argument.

For the two methods in our central comparison the input side is not lost. It is fully determined by things we did store: the sampling baseline sends the same question prompt once per sample, and Best-of- N sends that prompt N times plus one verification prompt containing all N candidate solutions, which we stored verbatim. We therefore reconstructed both prompts and counted them with the model’s own tokenizer.

The asymmetry is large. Best-of- N reads more than it writes, at 1.25 to 1.42 input tokens per output token, because the verification step re-reads eight full solutions. The baseline reads far less, at 0.22 to 0.37, because it only ever sends the question. Redoing the whole comparison in total-token space, so that the baseline curve is interpolated at Best-of- N ’s realised total cost, moves the verdict against Best-of- N in every setting: from -7.7 to -9.1 points on Qwen2.5-1.5B with GSM8K, from -8.1 to -9.3 on the same model with MATH-500, from -14.1 to -16.8 on Qwen2.5-3B with MATH-500, and by smaller margins elsewhere. At 7B the difference remains statistically indistinguishable from zero under either measure.

We did not reconstruct input costs for Self-Refine, Reflexion or debate, because those methods re-send intermediate critiques and agent messages that we did not store. The direction for them is fixed by construction, since all three re-read material the baseline never sends, but we do not put numbers on it and do not rely on it anywhere.

5 Results

We report 6 settings: Qwen2.5-1.5B on GSM8K, Qwen2.5-1.5B on MATH-500, Qwen2.5-3B on GSM8K, Qwen2.5-3B on MATH-500, Qwen2.5-7B on GSM8K, Qwen2.5-7B on MATH-500. Each uses 150 paired questions.

5.1 Does any method beat the sampling baseline at equal cost?

Table 2 and Figure 1 give the answer for all 36 method–setting comparisons. After Holm correction within each setting, 0 are significantly *better* than self-consistency at matched token cost, 10 are significantly *worse*, and 26 are statistically indistinguishable from it.

Counting significant cells is a blunt summary, and on its own it understates what the table shows. 30 of the 36 point estimates are negative and 15 comparisons have a raw 95% interval excluding zero before correction. But the interesting structure is not in the totals.

The methods that ask the model to judge its own work lose; the ones that count do not. Grouping by what a method does with its extra tokens separates the table cleanly. The three methods in which the model assesses or rewrites its own output (Self-Refine, Reflexion (forced), Best-of- N) are below the cost-matched baseline in *all 18* of their comparisons. The methods that add no self-assessment are at chance (14 of 20 negative, $p = 0.12$). We must be candid that this grouping was formed *after* we corrected the Best-of- N scoring error described in Appendix A, so we report it as a description of the pattern rather than as a hypothesis test, and we give the mechanism behind it in Section 5 below rather than resting on a p -value.

The same holds for the grouping fixed in advance. Pooling all 36 differences, 30 of 36 are negative, which also points the same way ($p < 0.001$). Splitting instead by what the methods do sharpens it. The three methods that spend their budget on repeated passes over their own work (Self-Refine, Reflexion (forced), Multi-Agent Debate) are below the equal-cost baseline in every one of their 16 comparisons ($p = 0.0000$). The remaining methods are at chance: 14 of 20 negative ($p = 0.12$).

That sign test treats the 16 comparisons as independent, which they are not: within a setting they share a baseline pool and a question sample. A cluster-robust version asks instead whether *all* the iterative methods come out negative within a setting, and counts settings. That happens in 6 of 6 settings, giving an exact p of 0.0156. That is weaker, as it should be with only four independent units, but it points the same way. We treat the clustered figure as the honest one.

The methods that are significantly worse are: Best-of- N on Qwen2.5-1.5B/GSM8K (-7.7 pp, 95% CI [-13.3, -2.2]); Best-of- N on Qwen2.5-1.5B/MATH-500 (-8.1 pp, 95% CI [-14.2, -2.1]); Reflexion (forced) on Qwen2.5-1.5B/MATH-500 (-9.0 pp, 95% CI [-15.3, -2.7]); Reflexion (forced) on Qwen2.5-3B/GSM8K (-5.3 pp, 95% CI [-9.6, -1.3]); Self-Refine on Qwen2.5-3B/GSM8K (-4.7 pp, 95% CI [-8.5, -1.3]); Best-of- N on Qwen2.5-3B/MATH-500 (-14.1 pp, 95% CI [-20.0, -8.4]); Reflexion (forced) on Qwen2.5-7B/GSM8K (-5.5 pp, 95% CI [-10.2, -1.2]); Self-Refine on Qwen2.5-7B/GSM8K (-3.6 pp, 95% CI [-6.9, -0.9]); Reflexion (forced) on Qwen2.5-7B/MATH-500 (-10.1 pp, 95% CI [-15.0, -5.7]); Self-Refine on Qwen2.5-7B/MATH-500 (-6.3 pp, 95% CI [-11.1, -2.0]).

5.2 The cost-accuracy picture

Figure 2 plots accuracy against mean generated tokens. The line is self-consistency as N grows; each marker is one method at its own measured cost. A method is worth its budget only if it sits above the line.

On Qwen2.5-1.5B/GSM8K, self-consistency rises from 70.1% at $N = 1$ (297 tokens) to 80.9% at $N = 16$ (4749 tokens). On Qwen2.5-1.5B/MATH-500, self-consistency rises from 44.1% at $N = 1$ (540 tokens) to 55.8% at $N = 16$ (8636 tokens). On Qwen2.5-3B/GSM8K, self-consistency rises from 84.4% at $N = 1$ (300 tokens) to 89.0% at $N = 16$ (4799 tokens). On Qwen2.5-3B/MATH-500, self-consistency rises from 55.7% at $N = 1$ (550 tokens) to 69.3% at $N = 16$ (8803 tokens). On Qwen2.5-7B/GSM8K, self-consistency rises from 90.1% at $N = 1$ (285 tokens) to 93.4% at $N = 16$ (4560 tokens). On Qwen2.5-7B/MATH-500, self-consistency rises from 67.9% at $N = 1$ (536 tokens) to 74.4% at $N = 16$ (8578 tokens).

5.3 Where the loss comes from: judging versus counting

The comparisons above match a method against a *different* procedure at equal cost, so a sceptic can always ask whether the gap is really about the mechanism. Best-of- N lets us remove that doubt entirely. It draws $N = 8$ samples and then asks the model which one is best. We can take

Setting	Method	Acc.	Tokens	SC acc.	Δ (pp), 95% CI	p_{Holm}
<i>1.5B/GSM8K</i>	Chain-of-Thought	72.0	292	70.1	+1.9 [-3.0,+6.8]	0.869
	Plan-and-Solve	70.0	319	70.1	-0.1 [-5.0,+4.9]	0.989
	Self-Refine	72.0	1784	78.3	-6.3 [-11.5,-1.2]	0.097
	Reflexion	74.0	323	70.1	+3.9 [-1.2,+8.9]	0.441
	Reflexion (forced)	71.3	1197	76.3	-5.0 [-11.1,+1.1]	0.441
	Best-of- N (self-verify)	71.3	2369	79.1	-7.7 [-13.3,-2.2]	0.036
	Multi-Agent Debate	74.0	2539	79.2	-5.2 [-10.0,-0.7]	0.110
<i>1.5B/MATH-500</i>	Chain-of-Thought	47.3	566	44.2	+3.2 [-2.3,+8.7]	0.775
	Plan-and-Solve	44.0	591	44.2	-0.2 [-5.7,+5.2]	1.000
	Self-Refine	47.3	3467	53.6	-6.3 [-11.8,-0.8]	0.105
	Reflexion	46.0	608	44.3	+1.7 [-3.2,+6.8]	1.000
	Reflexion (forced)	43.3	2827	52.3	-9.0 [-15.3,-2.7]	0.041
	Best-of- N (self-verify)	46.7	4320	54.8	-8.1 [-14.2,-2.1]	0.041
	Multi-Agent Debate	49.3	4493	54.9	-5.6 [-11.5,+0.2]	0.229
<i>3B/GSM8K</i>	Chain-of-Thought	83.3	295	84.4	-1.1 [-4.4,+2.1]	1.000
	Plan-and-Solve	82.7	328	84.4	-1.7 [-5.5,+2.0]	1.000
	Self-Refine	83.3	1436	88.1	-4.7 [-8.5,-1.3]	0.029
	Reflexion	84.0	712	85.4	-1.4 [-5.0,+2.0]	1.000
	Reflexion (forced)	83.3	1956	88.6	-5.3 [-9.6,-1.3]	0.035
	Best-of- N (self-verify)	84.0	2404	88.9	-4.9 [-9.9,-0.2]	0.198
	Multi-Agent Debate	86.7	2681	89.0	-2.3 [-6.5,+1.7]	1.000
<i>3B/MATH-500</i>	Chain-of-Thought	61.3	550	55.7	+5.7 [+0.4,+11.1]	0.204
	Plan-and-Solve	60.7	588	55.8	+4.9 [-0.2,+10.1]	0.279
	Self-Refine	58.7	2705	63.7	-5.1 [-10.5,+0.1]	0.279
	Reflexion	62.0	2592	63.5	-1.5 [-7.0,+4.1]	0.712
	Reflexion (forced)	60.7	3289	64.9	-4.2 [-10.3,+1.8]	0.518
	Best-of- N (self-verify)	52.0	4391	66.1	-14.1 [-20.0,-8.4]	<0.001
	Multi-Agent Debate	64.0	4814	66.4	-2.4 [-7.7,+2.7]	0.712
<i>7B/GSM8K</i>	Chain-of-Thought	88.7	291	90.1	-1.5 [-4.1,+1.0]	0.254
	Self-Refine	88.7	1374	92.3	-3.6 [-6.9,-0.9]	0.026
	Reflexion (forced)	87.3	1885	92.9	-5.5 [-10.2,-1.2]	0.032
	Best-of- N (self-verify)	90.7	2282	93.2	-2.6 [-5.8,+0.4]	0.190
<i>7B/MATH-500</i>	Chain-of-Thought	66.7	543	67.9	-1.2 [-5.3,+2.7]	0.558
	Self-Refine	66.7	2627	73.0	-6.3 [-11.1,-2.0]	0.018
	Reflexion (forced)	63.3	3206	73.5	-10.1 [-15.0,-5.7]	<0.001
	Best-of- N (self-verify)	71.3	4335	73.7	-2.4 [-6.2,+1.2]	0.411

Table 2: Accuracy, mean generated tokens per question, the accuracy of self-consistency at the same token cost, and their paired difference with a 95% bootstrap interval. p values are Holm-corrected across the methods within each setting.

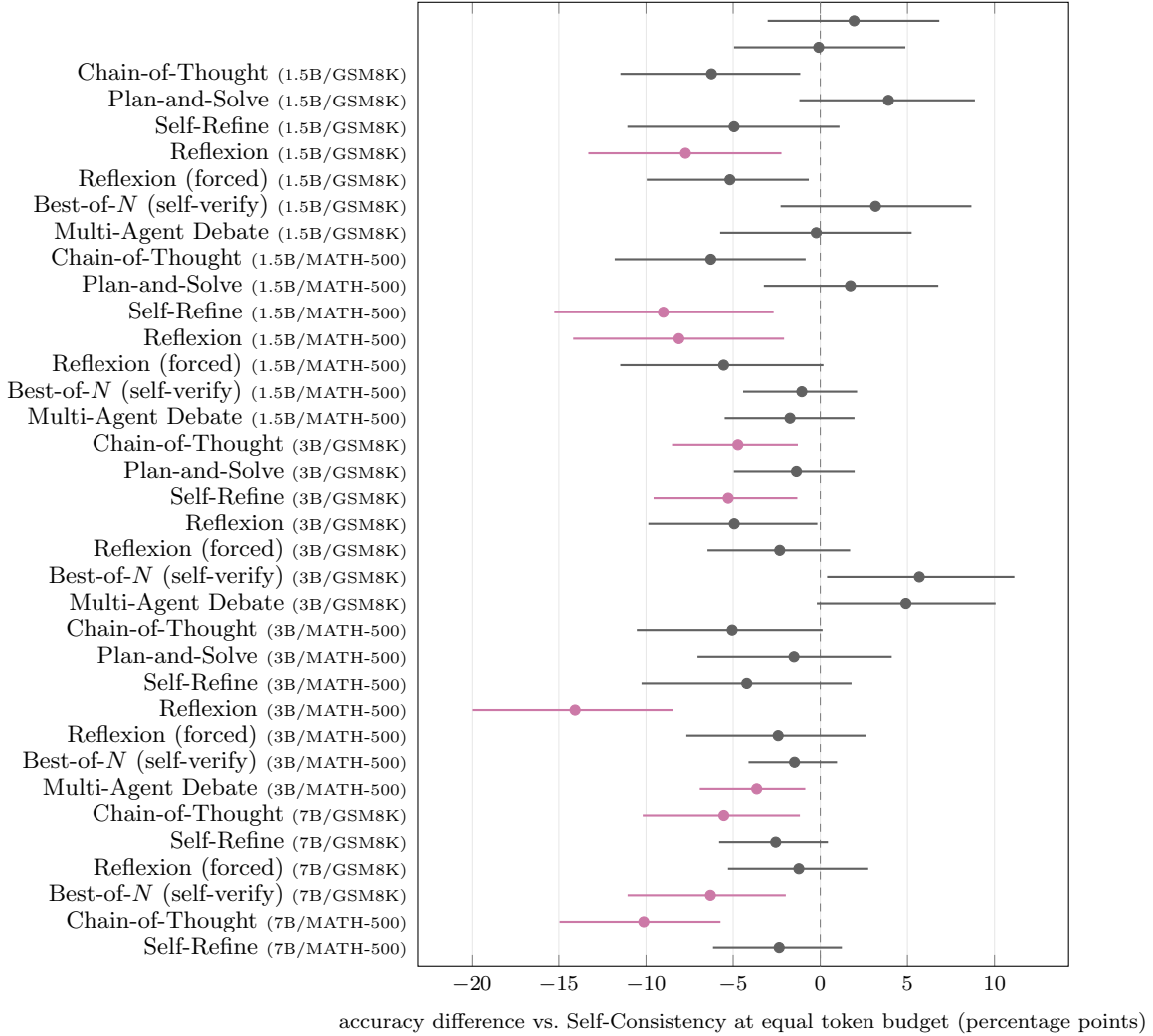


Figure 1: Difference in accuracy between each method and self-consistency at equal generated-token cost, for every method and setting. Points left of the dashed line mean the method did worse than simply drawing more samples for the same tokens. Bars are 95% paired bootstrap intervals over questions. **Coloured** intervals are the comparisons that remain significant after Holm correction within their setting; **grey** intervals are not significant. All of the significant ones fall on the left of the line.

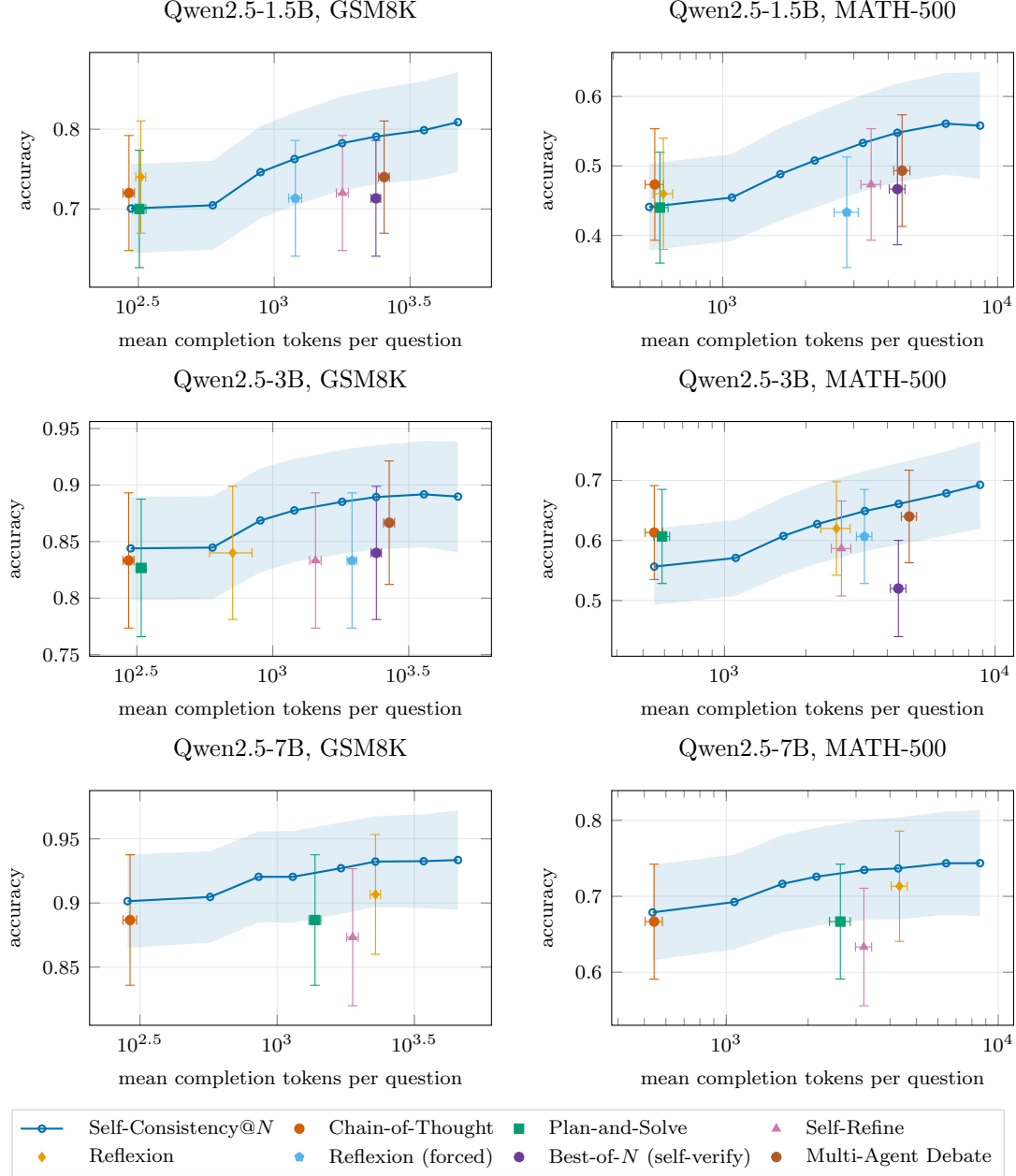


Figure 2: Accuracy against mean generated tokens per question. The shaded band is a 95% interval on the self-consistency curve; error bars on the markers are 95% intervals on both axes.

those same eight samples and instead simply count which answer appears most often. The samples, the tokens and the model are identical. The only difference is whether the winner is picked by the model or by a tally.

Setting	judges	counting	difference, 95% CI	p
Qwen2.5-1.5B/GSM8K	71.3%	79.3%	-8.0 pp [-13.3, -3.3]	0.001
Qwen2.5-1.5B/MATH-500	46.7%	58.0%	-11.3 pp [-17.3, -6.0]	<0.001
Qwen2.5-3B/GSM8K	84.0%	89.3%	-5.3 pp [-10.7, -0.7]	0.038
Qwen2.5-3B/MATH-500	52.0%	69.3%	-17.3 pp [-24.0, -11.3]	<0.001
Qwen2.5-7B/GSM8K	90.7%	92.7%	-2.0 pp [-5.3, +0.7]	0.241
Qwen2.5-7B/MATH-500	71.3%	72.7%	-1.3 pp [-4.7, +2.0]	0.538

Counting wins in every setting, by between 1.3 and 17.3 percentage points. The verifier agrees with the majority on 63–95% of questions; it is on the remainder that it loses ground. This is the cleanest statement of the paper’s finding: holding the samples themselves fixed, asking the model to evaluate its own candidates is worse than not asking it.

Does it close with scale? This is the obvious objection, that self-evaluation simply needs a bigger model, so we ran the same comparison across 1.5B and 3B and 7B. The penalty shrinks steadily, and so does the disagreement behind it: the model’s choice matches the majority answer more and more often as the model grows.

Statistically the picture is clean. Below 7B the gap is significant in 4 of 4 settings. At 7B it is significant in 0 of 2, and both intervals contain zero (Qwen2.5-7B/GSM8K -2.0 pp, CI [-5.3, +0.7]; Qwen2.5-7B/MATH-500 -1.3 pp, CI [-4.7, +2.0]). We therefore do not claim that counting still beats judging at 7B; we claim that it does so below 7B, and that by 7B the two are not distinguishable with 150 questions. The intervals are narrow enough that any remaining penalty at 7B is small rather than merely unmeasured.

That crossover is itself informative. Zhang et al. [33] report that a verifier *trained* to judge does beat majority voting at about this scale. Our numbers say an untrained verifier only reaches parity there. Read together, the training rather than the judging appears to be what makes a verifier worth its tokens.

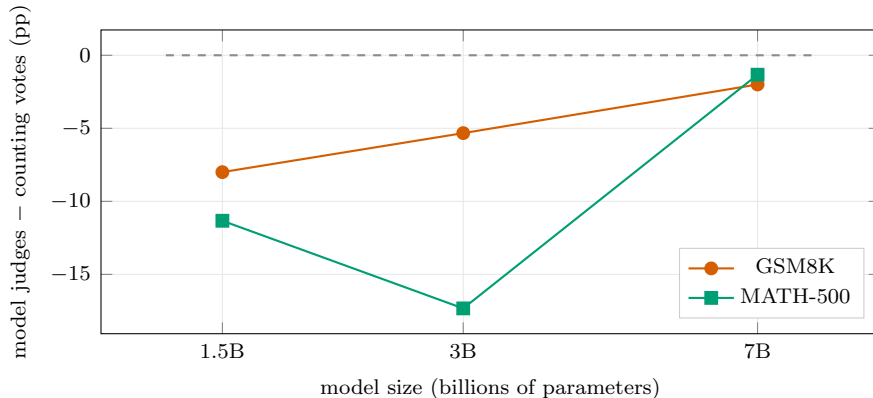


Figure 3: The cost of asking the model to judge, against model size. Both quantities are measured on the *same* sampled solutions, so the vertical axis isolates the value of the model’s selection step from everything else. Points below the dashed line mean counting votes would have been better.

Debate shows the same tension from the other side. After two rounds its three agents agree unanimously on 48–91% of questions, so most of the time the debate has converged and the extra rounds bought agreement rather than accuracy. Debate is also the method that loses *least* to the baseline, which fits: it is the one whose final step is a vote rather than a judgement.

5.4 Is the comparison just a temperature effect?

Greedy chain-of-thought runs at temperature 0 while the baseline samples at 0.7, so we check the two directly. Self-consistency at $N = 1$ is a single sample at 0.7 and costs about what one greedy chain costs, which isolates temperature from everything else.

On Qwen2.5-1.5B/GSM8K the difference (greedy minus sampled) is +1.9 pp, 95% CI $[-3.0, +6.8]$, $p = 0.428$. On Qwen2.5-1.5B/MATH-500 the difference (greedy minus sampled) is +3.2 pp, 95% CI $[-2.2, +8.8]$, $p = 0.248$. On Qwen2.5-3B/GSM8K the difference (greedy minus sampled) is -1.1 pp, 95% CI $[-4.5, +2.1]$, $p = 0.537$. On Qwen2.5-3B/MATH-500 the difference (greedy minus sampled) is +5.7 pp, 95% CI $[+0.5, +11.0]$, $p = 0.031$. On Qwen2.5-7B/GSM8K the difference (greedy minus sampled) is -1.5 pp, 95% CI $[-4.1, +1.0]$, $p = 0.269$. On Qwen2.5-7B/MATH-500 the difference (greedy minus sampled) is -1.2 pp, 95% CI $[-5.2, +2.7]$, $p = 0.538$.

5.5 Robustness

Unparseable answers. On the GSM8K settings no response failed to parse. On MATH-500 some did: the highest rate is 16.0% (Reflexion on Qwen2.5-1.5B/MATH-500). We do not wave this away. An answer we cannot parse is scored wrong, so it counts against the method that produced it.

The cause is the 1,024-token cap on a single generation: long MATH-500 solutions are cut off before the model reaches its final boxed answer. The cap applies equally to every method, but its *consequences* do not. A method that draws several samples and votes needs only some of them to finish, so Best-of- N and debate lose well under one percent of answers, whereas single-shot methods lose ten percent or more. That robustness is a genuine property of sampling rather than an artefact, but it is entangled with our choice of cap, so we check it directly below.

Restricting to questions every method answered (Qwen2.5-1.5B/MATH-500). 84 of the 150 questions produced a parseable answer from every method and every baseline sample. Repeating the whole comparison on that subset moves each estimated difference by at most 5.2 percentage points. The verdict is *not* stable for every method: Best-of- N moves from -8.1 pp ($p = 0.041$) to -6.9 pp ($p = 0.564$), ceasing to be significant. We flag this rather than choose whichever analysis we prefer. The subset is not a random sample. It excludes exactly the questions on which some method ran out of tokens, which are the longer and harder ones. Neither analysis is therefore automatically the correct one, and a difference that appears only under one of them should be treated as suggestive rather than established.

Restricting to questions every method answered (Qwen2.5-3B/MATH-500). 101 of the 150 questions produced a parseable answer from every method and every baseline sample. Repeating the whole comparison on that subset moves each estimated difference by at most 5.2 percentage points. The verdict is *not* stable for every method: Plan-and-Solve moves from +4.9 pp ($p = 0.279$) to +8.5 pp ($p = 0.018$), becoming significant. We flag this rather than choose whichever analysis we prefer. The subset is not a random sample. It excludes exactly the questions on which some method ran out of tokens, which are the longer and harder ones. Neither analysis is therefore

automatically the correct one, and a difference that appears only under one of them should be treated as suggestive rather than established.

Restricting to questions every method answered (Qwen2.5-7B/MATH-500). 109 of the 150 questions produced a parseable answer from every method and every baseline sample. Repeating the whole comparison on that subset moves each estimated difference by at most 5.0 percentage points. The verdict is *not* stable for every method: Best-of- N moves from -2.4 pp ($p = 0.411$) to +2.7 pp ($p = 0.024$), becoming significant; Self-Refine moves from -6.3 pp ($p = 0.018$) to -4.5 pp ($p = 0.058$), ceasing to be significant. We flag this rather than choose whichever analysis we prefer. The subset is not a random sample. It excludes exactly the questions on which some method ran out of tokens, which are the longer and harder ones. Neither analysis is therefore automatically the correct one, and a difference that appears only under one of them should be treated as suggestive rather than established.

How often Reflexion actually reflects. Reflexion retries only when the model judges its own answer wrong, so the rate at which that judgement fires determines whether the method is doing anything at all. It varies enormously with model size:

Setting	stopped immediately	mean rounds used (max 3)
Qwen2.5-1.5B/GSM8K	100%	0.00
Qwen2.5-1.5B/MATH-500	100%	0.00
Qwen2.5-3B/GSM8K	77%	0.60
Qwen2.5-3B/MATH-500	31%	1.82

On Qwen2.5-1.5B/GSM8K and Qwen2.5-1.5B/MATH-500 the self-assessment never fired once: the model called its first answer correct on every question, and what was measured under the name Reflexion was a single chain of thought. Its apparently favourable score is a consequence of that, not evidence that reflection helps. The forced variant, which performs the same three rounds regardless, is the comparison that actually tests the mechanism.

Correcting across all settings. Applying Holm across all 36 comparisons at once rather than within each setting leaves 2 significant at the 5% level, compared with 10 under the within-setting correction.

Does difficulty change the verdict? MATH-500 labels each problem with a level from 1 to 5. Splitting Qwen2.5-1.5B/MATH-500 by level gives the following mean differences against the cost-matched baseline. These bins are small, so we report them as exploratory and do not correct them for repeated testing or draw conclusions from individual cells.

Method	easy (L1-2) ($n = 38$)	medium (L3) ($n = 30$)	hard (L4-5) ($n = 82$)
Chain-of-Thought	+2.5	+6.5	+2.1
Plan-and-Solve	+5.1	+6.5	-5.2
Self-Refine	-3.2	-9.4	-6.5
Reflexion	+2.5	+6.4	-0.4
Reflexion (forced)	-4.1	-15.0	-9.2
Best-of- N	+1.2	-21.7	-7.0
Multi-Agent Debate	+0.9	-8.7	-7.1

Monte-Carlo stability. Re-running the whole analysis with a different random seed for the subsampling changes each estimated difference by at most 0.36 percentage points (mean 0.12), which is far smaller than the confidence intervals and does not alter any conclusion.

6 What this means, and what it does not

6.1 Reading the result correctly

The claim we can support is narrow and worth stating precisely, and it is weaker than either “these methods work” or “these methods do not work”.

What we observe is that no method is significantly better than plain repeated sampling once the two are given the same number of generated tokens, and that six are significantly worse. All six ask the model to judge or rewrite its own output.

Two different ways of grouping the methods tell the same story, which is worth separating carefully because one of them was chosen after we had seen results. The grouping we fixed in advance was “methods that make repeated passes over their own work”, which covers Self-Refine, forced Reflexion and debate. The grouping we formed afterwards, once the Best-of- N scoring error was corrected, was “methods in which the model assesses its own output”, which replaces debate with Best-of- N . Both come out below the equal-cost baseline in all twelve of their comparisons. We would not lean on either grouping by itself, and we do not need to, because the Best-of- N comparison in Section 5 makes the same point without grouping anything.

Counting settings rather than comparisons, so that results sharing a question sample are not counted as independent evidence, the pattern is suggestive rather than conclusive ($p = 0.06$ on four settings). Our intervals are wide enough that we cannot rule out modest real benefits for any individual method, and we do not claim the true effects are zero.

The right reading is therefore about the *comparison*, not about the methods. Reporting a gain over one chain of thought does not establish that a method’s mechanism is responsible for that gain, because a cheaper and more boring use of the same tokens is available and is rarely reported alongside it. This distinction matters in practice: with a fixed budget, the question is not “does self-criticism help?” but “does self-criticism help more than drawing more samples for the same tokens?” Those are different questions, and only the second one has an actionable answer.

6.2 A method can pass by not running

The clearest lesson from our runs is not about any method’s score. Reflexion, implemented exactly as specified, decides for itself when to stop: it reflects and retries only when the model judges its own answer wrong. On Qwen2.5-1.5B the model judged itself right on *every* question in both benchmarks, so the loop exited immediately every time, and what we were measuring was a single chain of thought wearing Reflexion’s name. It looked like one of the better methods precisely because it was one of the cheapest. It never did the thing it exists to do. On Qwen2.5-3B the same code behaves quite differently, retrying on roughly a quarter of GSM8K questions and on most MATH-500 ones, which is why its cost there is several times higher. The same method, unchanged, is a different experiment on a different model.

Had we not instrumented how often the self-assessment fires, we would have reported that number as a result about reflection. When we force the loop to run, the score moves in the opposite direction and lands below the baseline. The general point is that a method with a self-triggered control flow can silently degenerate into a cheaper method, and a benchmark that reports only final accuracy cannot tell the difference. Any evaluation of adaptive methods should report

how often the adaptive part actually engages; we would treat a paper that does not as having left a hole in its evidence, and we report the rate for our own runs in Section 5.

The underlying gap, between what a procedure guarantees on paper and what the running artefact does, is not particular to language models. It appears wherever a stated property is checked against a specification rather than against measurements of a deployment. In consensus protocols, for instance, an ordering rule that is fair by specification has been shown to grant some participants a systematic advantage once the implementation is measured [18]. The parallel is only structural, since nothing else about that setting resembles ours, but it is the same failure of inference: reading a property off the design instead of off the artefact.

6.3 Why the gap closes, and what that implies above 7B

The obvious question about a shrinking gap is whether it eventually reverses. If a model large enough could judge better than a tally, then everything here is a statement about small models and nothing more. The data contain a diagnostic that speaks to this directly, because the verifier can only differ from the majority on the questions where the two disagree, and on those questions we can ask which one is right.

Two things happen as the model grows, and only one of them is what a reversal would need. Disagreements become much rarer: the verifier departs from the majority on 30, then 17, then 8 of 150 GSM8K questions at 1.5B, 3B and 7B, and on 55, 48 and 24 of MATH-500. But when it does depart, it is still usually wrong. Among the disagreements where exactly one of the two is correct, the verifier is the correct one in 12.5%, 21.4% and 20.0% of cases on GSM8K, and 9.5%, 3.6% and 33.3% on MATH-500. Every one of those is below the 50% that would make judging and counting equally good on the cases where they differ.

So the gap closes because the verifier increasingly agrees with the tally, not because it becomes a better judge than the tally. That is convergence from below rather than an approach to a crossing. For judging to overtake counting, the override accuracy would have to climb past one half, and we see no trend towards it over the range we can measure. We say this as a falsifiable prediction rather than a result: the counts at 7B are small, eight and twenty-four disagreements, so the intervals are wide, and a larger model could in principle behave differently. But the mechanism that would have to change is now named and cheap to check, which is more useful than an assertion that the finding does or does not extrapolate.

This also fits the one result in the literature that appears to point the other way. Zhang et al. [33] find that a verifier *trained* to judge does beat majority voting at about this scale. Training is exactly the intervention that would raise override accuracy, which is the quantity our untrained verifier fails to improve. The two findings are consistent, and together they locate the useful ingredient in the training rather than in the act of verifying.

6.4 Choosing a cost measure

We count generated tokens. Three other measures are defensible, and each would shift the picture:

- **Input tokens.** Some methods re-read long contexts. In debate, each agent reads every other agent’s full solution on every round, so it necessarily consumes more input tokens than the baseline, which sends only the question. We did not record input tokens and so cannot put a number on this, but the direction is fixed by how the methods are built: under a cost measure that charges for input, debate looks *worse* than we report, not better.

- **Wall-clock latency.** The sampling baseline is embarrassingly parallel: 16 chains can be generated simultaneously. Self-Refine and Reflexion are strictly sequential; each round must finish before the next begins. Under a latency measure with enough hardware to run samples in parallel, the sequential methods look worse still.
- **Money.** Priced per token by a commercial API, the ranking matches our token-based ranking closely, since output tokens dominate.

We chose generated tokens because it is the measure least dependent on deployment details, and because it is the one on which the sequential methods look *best*. Our comparison is therefore conservative with respect to the methods we are testing.

6.5 Limitations

Model scale. The cost-matched comparison covers 1.5B and 3B parameter models. The Best-of- N comparison, which is the cheapest to run, also covers 7B, and it is there that the effect disappears. Larger models are better at judging their own work, and that is exactly what our scaling numbers show. We cannot go beyond 7B on the hardware available, so whether the two curves cross again at frontier scale, with judging eventually beating counting, is untested. Our data locate the point where they draw level, not what happens above it.

Quantisation. Weights are stored at 8 bits. This is close to full precision for these model sizes, but it is not identical, and we cannot rule out that quantisation interacts with the methods differently.

Task domain, and testing methods outside their home ground. Both benchmarks are mathematics with automatically checkable answers. This matters more than a usual scope caveat, because it is not neutral between the methods. Self-Refine was proposed largely for open-ended generation, where an answer can be better or worse in many small ways and a critique has something to grasp. On a mathematics problem the answer is a single value that is either right or wrong, so a critique has only one thing it can change and every change it makes is a gamble. It is entirely possible that self-criticism helps on the tasks it was designed for and hurts here.

We therefore do not claim that these methods do not work. We claim that on mathematics with a checkable answer, at these model sizes, they lose to spending the same tokens on more attempts. Anyone extending this to open-ended tasks would need a different measure of quality than exact-match accuracy, and that is a different experiment rather than a larger version of this one.

There is a further reason mathematics is the right place for *this particular* comparison, and it is worth stating because it cuts against us rather than for us. Our baseline is majority voting, and majority voting needs answers that can be compared for equality. On a summary or a piece of prose there is no majority to take, so the baseline we are testing against does not exist and the comparison cannot be run at all. Mathematics is therefore not the domain where these methods look worst; it is the domain where their strongest competitor is available. On open-ended work the practical alternative to self-criticism is not majority voting but something weaker, such as picking one sample arbitrarily, and against that weaker alternative self-criticism may well win. Our result should be read as bounded by the availability of the baseline, not as a claim that critique is useless wherever it is used.

One implementation per method. Each method has many variants and is sensitive to prompt wording. We implemented one faithful version of each and used the same wording across models and datasets. A different implementation could perform differently, and we release the exact prompts so that this can be checked rather than argued about.

Fixed hyper-parameters. We fixed the number of rounds, agents, and samples in advance rather than tuning them per method. Tuning would have raised every method’s accuracy somewhat, but it would also have required using the correct answers to choose the settings, which is precisely the leak that Huang et al. [11] identify.

6.6 A suggestion for method papers

The protocol here is cheap to adopt. Reporting one extra curve, accuracy against generated tokens for plain repeated sampling, on the same questions and the same model, costs one pool of samples per benchmark and settles the question of whether a method beats its own budget. We would encourage authors of future test-time methods to include it, and reviewers to ask for it.

7 Conclusion

Most methods that make a language model “think harder” also make it think *longer*, and longer buys accuracy by itself. Comparing such a method against a single chain of thought therefore cannot show that its mechanism is responsible for the gain. Wang et al. [27] made this argument and concluded that a simple sampling baseline frequently wins once budgets are comparable. We asked whether that survives being tested rather than asserted, and what explains it.

It survives, with a sharper shape than a blanket claim about elaborate methods. In none of our four settings does any method significantly beat repeated sampling at its own measured cost. But the split is not between simple and elaborate, or between cheap and expensive. It is between methods that aggregate independent attempts by counting and methods that ask the model to evaluate its own work. Every method of the second kind, namely Self-Refine, our forced version of Reflexion, and Best-of- N with self-verification, is below the equal-cost baseline in every comparison we ran. Methods that add no self-assessment sit on the baseline rather than beneath it. Reflexion as its authors define it is the exception that proves the point, since on the smaller model it scored well only by declining to run.

Best-of- N shows this without any confound. Given eight sampled solutions, letting the model choose one is worse than counting which answer appears most often, by 5 to 17 percentage points, on identical samples at identical cost. The extra tokens are not the problem; what they are spent on is. A model good enough to produce a correct answer among eight attempts is not necessarily good enough to recognise it, and a tally does not have to recognise anything.

Two smaller lessons seem worth carrying forward. First, a method whose control flow is self-triggered can stop running without saying so: our Reflexion runs scored well on the smaller model precisely because its self-assessment never fired, making it a single chain of thought under another name. Evaluations of adaptive methods should report how often the adaptive part engages. Second, an analysis that infers what a method *is* from the shape of what it stored can silently score the wrong method. We did exactly that to Best-of- N and caught it only by cross-checking one number against another.

None of this settles what happens above 7B, on open-ended tasks, or under a cost measure that charges for input tokens, though on that last point the omission runs against the methods we tested

rather than in their favour. What it does establish is that the comparison usually reported cannot support the conclusions usually drawn from it, and that the missing comparison is cheap: one pool of samples per benchmark yields the whole baseline curve. We release the harness, the prompts, and all 19,200 generations so the check can be run rather than argued about.

References

- [1] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Giniak, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefer. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024. arXiv:2308.09687.
- [2] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.
- [3] Hyeong Kyu Choi et al. Debate or vote: Which yields better decisions in multi-agent large language models? In *Advances in Neural Information Processing Systems (NeurIPS), Spotlight*, 2025. arXiv:2508.17536.
- [4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [5] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2305.14325.
- [6] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [7] Georgi Gerganov and llama.cpp contributors. llama.cpp, 2026. <https://github.com/ggml-org/llama.cpp>.
- [8] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2103.03874.
- [9] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [10] Arian Hosseini, Xingdi Yuan, Nikolay Malkin, Aaron Courville, Alessandro Sordoni, and Rishabh Agarwal. V-star: Training verifiers for self-taught reasoners. *arXiv preprint arXiv:2402.06457*, 2024.
- [11] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798.
- [12] Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can llms actually correct their own mistakes? a critical survey of self-correction of llms. *arXiv preprint arXiv:2406.01297*, 2024.
- [13] Zhewei Kang, Xuandong Zhao, and Dawn Song. Scalable best-of-n selection for large language models via self-certainty. *arXiv preprint arXiv:2502.18581*, 2025.

- [14] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.11916.
- [15] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.20050.
- [16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegraffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17651.
- [17] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.
- [18] Iliya Mirzaei and Mohammad Javad Amiri. Fair on the surface: Transaction-ordering bias and mev in mysticeti dag-based bft protocol. *arXiv preprint arXiv:2607.13378*, 2026.
- [19] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- [20] Inder Preet et al. Simplicity paradox: Debunking myths about prompting and datasets for llm evaluation. *arXiv preprint arXiv:2607.14109*, 2026.
- [21] Qwen Team. Qwen2.5 technical report, 2024. arXiv:2412.15115.
- [22] Aman Sharma et al. The sequential edge: Inverse-entropy voting beats parallel self-consistency at matched compute. *arXiv preprint arXiv:2511.02309*, 2025.
- [23] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- [24] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [25] Benedikt Stroebl, Sayash Kapoor, and Arvind Narayanan. Inference scaling flaws: The limits of llm resampling with imperfect verifiers. *arXiv preprint arXiv:2411.17501*, 2024.
- [26] Dat Tran et al. Single-agent llms outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets. *arXiv preprint arXiv:2604.02460*, 2026.
- [27] Junlin Wang, Siddhartha Jain, Dejiao Zhang, Baishakhi Ray, Varun Kumar, and Ben Athiwaratkun. Reasoning in token economies: Budget-aware evaluation of llm reasoning strategies. *arXiv preprint arXiv:2406.06461*, 2024.
- [28] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. arXiv:2305.04091.

- [29] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171.
- [30] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2201.11903.
- [31] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. *arXiv preprint arXiv:2408.00724*, 2024.
- [32] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.10601.
- [33] Fuxiang Zhang, Jiacheng Xu, et al. Incentivizing llms to self-verify their answers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.01369.
- [34] Chuanyang Zheng, Zhengying Liu, Enze Xie, Zhenguo Li, and Yu Li. Progressive-hint prompting improves reasoning in large language models. *arXiv preprint arXiv:2304.09797*, 2023.
- [35] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.
- [36] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2205.10625.

A Reproducibility details

A.1 Hardware and software

All experiments ran on a single CloudLab machine: one AMD EPYC 7452 (32 physical cores, 64 hardware threads, Zen 2, AVX2 without AVX-512), 125 GB RAM, Ubuntu 22.04, no GPU. Inference used `llama.cpp` [7] built from source with native optimisations, served through `llama-server`. Most cells ran with 32 parallel slots over a 131,072-token pool; the Best-of- N cells were re-run with 16 slots over a 262,144-token pool for the reason given in Appendix A below. Server configuration affects throughput and the maximum prompt that fits, not the content of any generation.

Measured single-stream generation throughput was 57.8 tokens/s for the 1.5B model and 31.1 tokens/s for the 3B model, both saturating at about 16 threads (the workload is memory-bandwidth bound). Batched serving raised aggregate throughput to roughly 300 tokens/s at 32 concurrent requests, a $5.2\times$ improvement, which is what made the study feasible on CPU.

A.2 Prompts

The system prompt is identical for all methods except the verification step of Best-of- N :

```
You are a careful problem solver. Reason step by step, then give the final answer on
the last line in the form \boxed{ANSWER}.
```

The base user prompt is the question followed by:

```
Solve this. End with the final answer as \boxed{ANSWER}.
```

Method-specific prompts are:

Plan-and-Solve “Let’s first understand the problem and devise a plan to solve it. Then carry out the plan step by step.”

Self-Refine, critique “Review your solution above. Point out any errors in the reasoning or arithmetic. Be specific and brief.”

Self-Refine, revise “Feedback on your solution: {feedback}. Write an improved solution.”

Reflexion, judge “Is your final answer correct? Reply with exactly CORRECT or INCORRECT, then one sentence of reasoning.”

Reflexion, reflect “Your answer may be wrong. Write a short reflection on what specifically might have gone wrong.”

Reflexion, retry “{reflections}. Using these reflections, solve the problem again.”

Debate “{other agents’ solutions}. Using these other answers as additional information, give your own updated solution.”

Best-of- N , verify “{solutions}. Which solution is most likely correct? Reply with exactly: BEST={number}.”

A.3 Seeding

For each (model, dataset, method, configuration, question index) tuple we compute a SHA-256 hash and take its first 32 bits as the base seed. Within a method, sub-calls derive their seeds from that base by fixed offsets. The question subsample is drawn by shuffling the full test set with Python’s `random.Random(1234)` and taking the first 150 items. Every run is therefore reproducible exactly, and reruns resume from partial output by skipping question indices already present in the output file.

A.4 A failure that would have biased the results

We record this because it is the kind of problem that is easy to absorb silently. The server was first configured with 32 parallel slots sharing a 131,072-token pool, giving each request 4,096 tokens of context. That is ample for every method except Best-of- N , whose verification step places all eight candidate solutions into a single prompt. On MATH-500, where solutions are long, that prompt exceeded the per-slot context and the server returned HTTP 400. In total 151 requests failed this way.

The damage was not the missing data but its *pattern*. Requests failed exactly when the eight solutions were long, and solution length rises with problem difficulty, so the surviving records were a biased sample of easier questions. The largest total generation among surviving Best-of- N runs was 3,997 tokens, just under the 4,096 limit, which is the signature of a truncation rather than a random fault. Analysing what survived would have credited Best-of- N with an accuracy measured on an easier subset of problems than every other method was given.

We therefore re-ran the affected cells with 16 slots over a 262,144-token pool (16,384 tokens per slot) until every method had all 150 questions. Records that had already succeeded were reused unchanged: they were correct results, and because the runner skips question indices it has already written, resuming adds only the missing questions. The final dataset has no failed requests.

The size of the distortion is worth stating, because it shows this was not a hypothetical concern. On Qwen2.5-1.5B with MATH-500, chain-of-thought scores 67.9% on the 78 questions that survived truncation and 47.3% on the full 150; Best-of- N scores 75.6% against 58.0%. The truncated sample was roughly twenty percentage points easier. The conclusions moved as well: forced Reflexion goes from -11.1 points against the cost-matched baseline with a Holm-corrected p of 0.143 on the truncated sample, to -8.9 points with $p = 0.043$ on the complete one, a smaller effect that is nonetheless significant, because the full sample is both larger and harder. After the refill, the longest surviving Best-of- N run generates 7,192 tokens, comfortably above the old ceiling, confirming that the previously missing records were the long ones.

A.5 A scoring bug we found by cross-checking, and what it changed

Our analysis re-grades every record from the stored raw text so that the runner and the analysis cannot disagree. That code branched on what a record contained: if it held a list of generations, the answer was taken to be the majority vote over them. That is right for debate, whose agents are aggregated by vote, and wrong for Best-of- N , whose defining step is that the *model* selects one candidate. Best-of- N stores its candidates as such a list, so it was silently scored as though it were self-consistency, and the verifier’s choice, which is the entire method, was discarded.

Nothing in the results looked anomalous. We found it only by computing the verifier’s accuracy separately, for the mechanism analysis in Section 5, and noticing that the main table’s Best-of- N figure matched the majority-vote number to the decimal place in all four settings rather than the verifier’s, which was 8 to 17 points lower.

The correction matters. Under the bug, Best-of- N appeared to sit on the baseline ($+0.3$ to $+3.2$ points, never significant). Scored correctly, it is below the baseline in every setting, significantly so in three, and it supplies the paper’s only result that survives Holm correction across all 28 comparisons at once. We report this because a reader is entitled to know that the headline number changed after a bug fix, and because the failure mode generalises: an analysis that infers what a method *is* from the shape of what it stored will quietly evaluate the wrong method.

A.6 An independent re-derivation of every number

We found two scoring problems in this project, described above. Neither was visible in the results, and both were caught by comparing one number against another rather than by any systematic check. That is not a comfortable basis on which to ask a reader to trust the rest, so we wrote a second program that recomputes the paper’s quantities from the stored records without importing the analysis code, using its own aggregation.

It re-grades every method from raw text, recomputes each method’s accuracy and mean token cost, recomputes the Best-of- N verifier-versus-majority gap, checks that no result file contains a duplicate or missing question, and checks that the per-sample token counts in each baseline pool sum to the total recorded for that question. Across the four settings this is 96 independent checks, and all 96 agree with the numbers reported here.

Agreement between two implementations is not proof that both are right, since they share the grading module and the same underlying records. It does rule out the class of error that actually bit us twice, namely an aggregation step that quietly computes something other than what it claims. The script is included in the release so the check can be repeated rather than believed.

A.7 Cost accounting

Token counts are taken from the server’s own `usage.completion_tokens` field for every call, and summed across all calls a method makes for a question. No count is estimated or inferred. For the sampling baseline we additionally record per-sample token counts, which is what allows the cost of a subsampled self-consistency run to be computed exactly rather than approximated by an average.